



NAME: SYED ABDUL BARI

REGISTRATION NUMBER: SP26-BAI-047

TOPIC: ASSIGNMENT 01

LAB TASK 01 Stakeholder and requirement analysis :

Stakeholder	Need
Student	Fair access to classrooms and labs.
Faculty members	Priority for scheduled teaching and urgent academic activity.
Administration	High room utilization
Security Staff	Traceable access records

REQUIREMENT CONFLICTS :

(i) Conflict 1 :

The security staff requires traceable access records while user needs his personal details to be kept private. From the given conditions of registration number, if a user other than security staff uses the Space allocation system, then his details must be kept private.

Solution :

The system should first take complete input from the user to identify from exactly what part of the University Hierarchy he belongs to like student, security, clerk, faculty etc. The system must show the availability of rooms or labs, but it must not show what is the reason for occupation of the room (Assigned registration number 0 or 1).

Policy:

The personal information must be kept private and restricted from public access. The system traces access details for security staff and for other users, it hides all the other fields showing room availability.

(ii) Conflict 2 :

Students are given access to resources like classrooms, laboratories and equipment but the faculty has the same needs keeping in mind the University's limited resources. This creates the problem of using the same equipment at the same time.

Solution :

Using the registration number (8 or 9), all the moveable and limited resources must be fed in the Space Allocation System. Keeping track of the number of apparatuses the availability can be managed. The person needing the equipment must give the necessary credentials as input to see whether he is a student or a faculty member.

Policy:

In case of a conflict between students and faculty, the faculty members will be given priority. Lab reservation and the equipment within the labs are different instance objects with different reservation.

(iii) Conflict 3 :

If a student has been given the responsibility of being assistant then his authority is greater than a student. But when he is not assisting then he is just a regular student. Changing his role can create certain conflicts in administrative and student user. If the student has a single identity he might be treated as a member of the executive board, or he might not be able to access the System as a teacher's assistant.

Solution :

The student must be given a second login credential For registration .When he is given a role of teacher assistant so that

he can do his administrative work and when he logs in as student the system does not consider him as an administrator.

Policy :

System must be designed in a way that it has role based login so it can manage students with multiple roles in the university.

Facts and Assumptions :

Facts :

- I) The Learning Space Allocation System will be used by students, faculty members, administration staff, security staff and maintenance staff.
- II) Rooms, laboratories, and equipment are scarce, so each requires a separate allocation.
- III) Security staff require traceable access records, while users expect personal information to remain private.
- IV) Assumptions :
- V) Role based login system.
- VI) Faculty is prioritized in the allocation of equipment.
- VII) Ordinary user see only the availability of the rooms while administration has full access to all audits.

LAB TASK 02 Object oriented conceptual model:

1. Class User:

Responsibility:

This class takes the necessary credentials, authenticates them and gives the person a specific role.

Methods:

public void Identification

public void Role;

2. Class Allocation:

Responsibility:

This class checks for the availability of a specific resource and checks its status. It checks precedence to avoid clashes.

Methods:

public void bookResources:

public void Precedencies:

3. Class Emergency:

Responsibility:

This class checks emergency cases and updates the audit. It also relocates the classrooms in case of updates

Methods:

public void Relocation:

public void updateAudit:

4. Class Records:

Responsibility:

This class traces records in case of security staff and prints the summary of information added.

Methods:

public void traceRecord:

public void printSummary:

5. Class Room:

Responsibility:

This class checks the availability of space in a room. It checks for chairs and sitting capacity. It also tells if the room is blocked for maintenance.

Methods:

public void Capacity:

public void chairs:

public void Maintenance:

public void reSchedule;

6. Class Role:

Responsibility: Defines permissions and booking limits. |

Attributes:

Methods:

public void roleName, public void priorityLevel.

7. Class moveableEquipment:**Responsibility:**

This class manages scarce resources and checks their issuance.

Methods:

public void checkAvailability

public void IssueEquip

- **Separation of Responsibilities & Avoiding Singular Controlling Class:**
- Avoiding a central class controlling the entire system is done by dividing the system into different classes, each performing their specific tasks. Each class performs their functions and hence the work of entire system is completed. The responsibilities are divided in the following way :
- **User (User Management):** Responsible only for remembering who the user is and what permissions they hold. It does not check room schedules or equipment inventory.

- **Room (Physical Space):** Responsible only for managing room details, like capacity and whether the room is physically available or under maintenance.
- **MovableEquipment (Resource Management):** Responsible only for tracking specific items (like lab kits or projectors) separately from the rooms.
- **Allocation (Booking Process):** Responsible only for creating the reservation link between a user and a space for a set time.
- **Record (Security & Privacy):** Responsible only for storing access logs and enforcing privacy rules when logs are requested.
- **EmergencyEvent (Audit Update):** Responsible only for handling building closures, relocations, and sending notifications
- **Administration(High room utilization):** Responsible for approving events and checking utilization of rooms.

In this way if one of the classes for example Room needs to be updated the rest of the classes remain intact.

LAB TASK 03 Abstraction, encapsulation and invariants:

Abstraction:

Class User:

A user enters his personal information such into the Spave management system. The Class takes required information from the user like student ID, registration number and role. This creates the opening sequence for the system to check, authenticate and give role to the user.

Class Allocation:

The allocation class stores the exact time slot and capturing its

approval state. Then it allots specific classrooms to the user based on availability. It manages the booking link between a user and a learning space based on availability.

Class Room:

The system abstracts the physical classroom into a digital object and manages the room using its room number and also checks the sitting capacity by methods Capacity() and Chairs().

Hidden Information, its operations and Ways to change it:

1. Class User:

Inside the User class, specific data fields like personalDetails, password, and userID are kept private to prevent unauthorized access. This class abstracts all other details of the student.

Operations:

Controlled operations on the personal details using the methods Identification() and Role() to specify type of user. This information can be changed by a new type of method updateProfile() ,

2. Class Allocation:

In the class allocation the user can see the availability of rooms but he cannot see the reason for unavailability of other rooms. It abstracts the real-world process of human negotiation and physical room occupation

Operations:

We can get a simple available or not available status using the methods in this class. Administrators can view the entire class history requesterID and reserving reasons. These can be changed by methods cancelReservation() and approveReservation().

3. Class Room

In this class the capacity and maintenance status are kept private so that someone can't change them directly.

It abstracts all the details of room from the user.

Operations:

The operations which can be performed are by the methods described in this class. We can check the sitting capacity in a class, whether it is blocked due to maintenance and the number of chairs available in the class. The status of a classroom can be changed by `setMaintenance()` and `fixCapacity()`.

Invariants :

The invariants in the system are:

1. Each room must have at least one student's sitting capacity and the method `capacity()` > 1 .
2. If a room's `Maintenance()` is true then it cannot be set to false .
3. The teachers are at higher precedence than the students.
4. An Allocation object's start time must be before its end time.

Encapsulation:

Class Room :

- **Hidden Data:** The status field (e.g., *Available*, *Maintenance*, *Emergency*) is kept private.
- **Operations:** It can only be modified through methods like `chairs()` or `Capacity()`, which verify the caller's role before updating the status.

Class Allocation (or Reservation):

- **Hidden Data:** The user's personal identity fields are kept private inside the object.
- **Operations:** A public method like `bookResources()` returns only a hidden identifier to standard users, revealing the full identity only when an authorized audit method is invoked

TASK 04 Task 4 Alternative designs and decision:

Attachable Roles: Keep one concrete User class and give it class Role hence roles get attached as objects instead of conversion into the type hierarchy.

1. **Overlapping Roles** — Subtyping breaks down the moment someone holds two roles at once: a student who also TAs would force multiple inheritance or a one-off StudentTA subclass. With attachable roles, you just add a Student role object and a TA role object to the same User — no new class required.
2. **Mid-Term Reassignment** : Under the hierarchy, moving someone from Student to Faculty means tearing down the old object and building a new one. Under attachable roles, you just swap the Role instance on the existing User at runtime.
3. **Structural Independence** : Subtype binds a person's identity directly to how a given role is implemented. Attachable roles keep identity and role logic fully separate.

TASK 05: Behaviour and edge cases

Handling Edge Cases

(i) Two simultaneous requests

If two simultaneous requests arise for a single room, the system checks the precedencies. If one of them is a teacher and the other is a student then the teacher is allocated the room. If two students request the same room, then they will be served on first com first serve basis.

(ii) Maintenance block after conrmation:

In this case, the system runs under the class Room and scheduled class is shifted to another available class using the reSchedule() method.

(iii) Emergency cancellation or relocation:

The class Emergency() shifts all the classes to available classes. It updates the audits and cancels classes in some cases.

TASK 06: Independent exploration :

Roll Based Access System:

Access control in this system follows Role-Based Access Control (RBAC): permissions attach to roles, not individual accounts. A user gains membership in one or more roles ("Student," "Administrator"), and actions such as "Book after hours" or "Override reservation" are authorized strictly at the role level. Because authorization logic never touches the individual user, administration stays scalable and secure (Sandhu et al., 1996).

This is precisely what makes concurrent, shifting roles tractable — a Student who picks up TA duties mid-semester needs no new user type. Implemented through composition, each User holds a list of Role objects; when an operation is requested, permission checks resolve to

the union of grants across every active role the user holds (Ferraiolo et al., 2001). That aggregation is what reconciles fluid academic standing with what would otherwise be static booking logic.

References:

1. Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *IEEE computer*, 29(2), 38-47.
2. Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., & Chandramouli, R. (2001). Proposed NIST standard for role-based access control. *ACM Transactions on Information and System Security (TISSEC)*, 4(3), 224-274.